MVLU
PRACTICAL NO:4

AIM: Digital Signatures:
Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

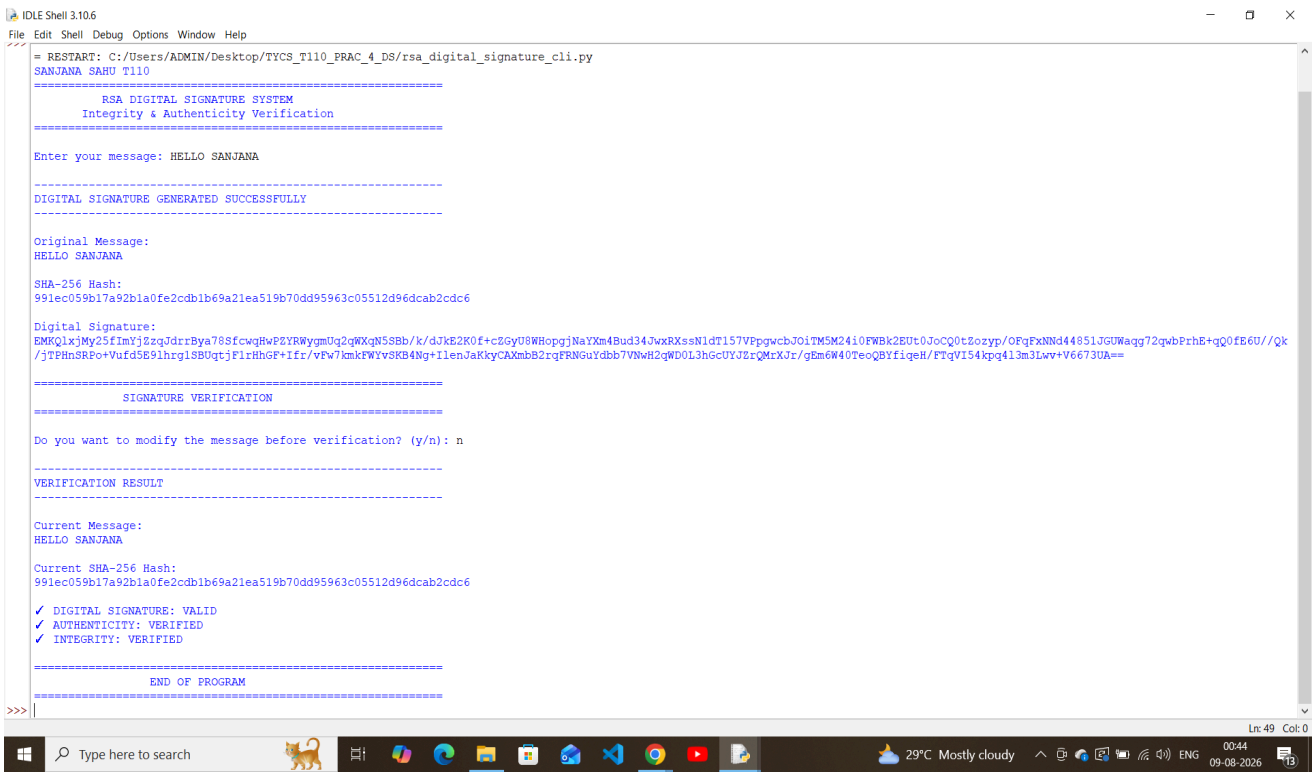
CLI CODE:

```
print("SANJANA SAHU T110")
# RSA DIGITAL SIGNATURE SYSTEM - CLI
# Integrity and Authenticity Verification

import hashlib
import base64

from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
# RSA KEY GENERATION
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

public_key = private_key.public_key()


# GENERATE DIGITAL SIGNATURE


def generate_signature(message):

    message_bytes = message.encode("utf-8")

    # SHA-256 Hash
    message_hash = hashlib.sha256(message_bytes).hexdigest()

    # RSA Private Key se Signature
    signature = private_key.sign(
        message_bytes,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    # Signature ko readable format mein convert
    signature_text = base64.b64encode(signature).decode("utf-8")

    return message_hash, signature_text

# VERIFY DIGITAL SIGNATURE

def verify_signature(message, signature_text):

    message_bytes = message.encode("utf-8")

    try:

        # Signature ko bytes mein convert
        signature = base64.b64decode(signature_text)

        # RSA Public Key se verification
        public_key.verify(
            signature,
            message_bytes,
            padding.PSS(
```

```
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    # Current message ka hash
    current_hash = hashlib.sha256(
        message_bytes
    ).hexdigest()

    return True, current_hash

  except Exception:

    current_hash = hashlib.sha256(
        message_bytes
    ).hexdigest()

    return False, current_hash

# MAIN PROGRAM

print("=" * 60)
print("         RSA DIGITAL SIGNATURE SYSTEM")
print("      Integrity & Authenticity Verification")
print("=" * 60)

# Enter message
message = input("\nEnter your message: ")

# Generate signature
original_hash, signature = generate_signature(message)

print("\n" + "-" * 60)
print("DIGITAL SIGNATURE GENERATED SUCCESSFULLY")
print("-" * 60)

print("\nOriginal Message:")
print(message)

print("\nSHA-256 Hash:")
print(original_hash)

print("\nDigital Signature:")
print(signature)
# VERIFICATION

print("\n" + "=" * 60)
print("            SIGNATURE VERIFICATION")
print("=" * 60)

choice = input(
  "\nDo you want to modify the message before verification? (y/n): "
)

if choice.lower() == "y":

  modified_message = input(
    "\nEnter modified message: "
  )

else:

  modified_message = message


# Verify
valid, current_hash = verify_signature(
  modified_message,
```

```
    signature
)


print("\n" + "-" * 60)
print("VERIFICATION RESULT")
print("-" * 60)

print("\nCurrent Message:")
print(modified_message)

print("\nCurrent SHA-256 Hash:")
print(current_hash)


if valid:

    print("\n✓ DIGITAL SIGNATURE: VALID")
    print("✓ AUTHENTICITY: VERIFIED")
    print("✓ INTEGRITY: VERIFIED")

else:

    print("\n✗ DIGITAL SIGNATURE: INVALID")
    print("✗ AUTHENTICITY: NOT VERIFIED")
    print("✗ INTEGRITY: NOT VERIFIED")


print("\n" + "=" * 60)
print("                END OF PROGRAM")
print("=" * 60)
```

OUTPUT:



GUI CODE:

```
print("SANJANA SAHU")
# DIGITAL SIGNATURE USING RSA
# Integrity + Authenticity Verification

import tkinter as tk
from tkinter import messagebox
import hashlib
import base64

from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
# RSA KEY GENERATION
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

public_key = private_key.public_key()

# FUNCTIONS
def generate_signature():

    message = message_entry.get("1.0", tk.END).strip()

    if not message:
        messagebox.showwarning(
            "Warning",
            "Please enter a message first!"
        )
        return

    try:
        # Convert message into bytes
        message_bytes = message.encode("utf-8")

        # Create digital signature using PRIVATE KEY
        signature = private_key.sign(
            message_bytes,
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH
            ),
            hashes.SHA256()
        )

        # Convert signature into readable text
        signature_text = base64.b64encode(signature).decode("utf-8")

        # Display signature
        signature_box.delete("1.0", tk.END)
        signature_box.insert(tk.END, signature_text)

        # Calculate original hash
        original_hash = hashlib.sha256(message_bytes).hexdigest()

        hash_box.delete("1.0", tk.END)
        hash_box.insert(tk.END, original_hash)

        status_label.config(
            text="✓ Digital Signature Generated Successfully",
            fg="green"
        )

    except Exception as e:
        messagebox.showerror("Error", str(e))
```

```
def verify_signature():

    message = message_entry.get("1.0", tk.END).strip()
    signature_text = signature_box.get("1.0", tk.END).strip()

    if not message:
        messagebox.showwarning(
            "Warning",
            "Please enter a message!"
        )
        return

    if not signature_text:
        messagebox.showwarning(
            "Warning",
            "Please generate a digital signature first!"
        )
        return

    try:
        # Convert signature back to bytes
        signature = base64.b64decode(signature_text)

        message_bytes = message.encode("utf-8")


        # VERIFY RSA SIGNATURE


        public_key.verify(
            signature,
            message_bytes,
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH
            ),
            hashes.SHA256()
        )


        # AUTHENTICITY + INTEGRITY SUCCESS


        current_hash = hashlib.sha256(message_bytes).hexdigest()

        current_hash_box.delete("1.0", tk.END)
        current_hash_box.insert(tk.END, current_hash)

        result_label.config(
            text="✓ SIGNATURE VALID\n\n"
                 "✓ AUTHENTICITY VERIFIED\n"
                 "✓ INTEGRITY VERIFIED",
            fg="green"
        )

        messagebox.showinfo(
            "Verification Successful",
            "Digital Signature is VALID!\n\n"
            "Authenticity: VERIFIED\n"
            "Integrity: VERIFIED"
        )

    except Exception:


        # MESSAGE WAS MODIFIED OR SIGNATURE IS INVALID


        current_hash = hashlib.sha256(
```

```
            message.encode("utf-8")
        ).hexdigest()

        current_hash_box.delete("1.0", tk.END)
        current_hash_box.insert(tk.END, current_hash)

        result_label.config(
            text="✗ SIGNATURE INVALID\n\n"
                 "✗ AUTHENTICITY NOT VERIFIED\n"
                 "✗ INTEGRITY NOT VERIFIED",
            fg="red"
        )

        messagebox.showerror(
            "Verification Failed",
            "Digital Signature is INVALID!\n\n"
            "Authenticity: NOT VERIFIED\n"
            "Integrity: NOT VERIFIED\n\n"
            "The message may have been modified."
        )


def clear_all():

    message_entry.delete("1.0", tk.END)
    signature_box.delete("1.0", tk.END)
    hash_box.delete("1.0", tk.END)
    current_hash_box.delete("1.0", tk.END)

    status_label.config(
        text="Ready",
        fg="blue"
    )

    result_label.config(
        text="Verification Result",
        fg="black"
    )

# GUI
root = tk.Tk()

root.title("RSA Digital Signature System")
root.geometry("950x750")
root.configure(bg="#EAF2F8")

# Allow resizing
root.resizable(True, True)
# TITLE

title = tk.Label(
    root,
    text="RSA DIGITAL SIGNATURE SYSTEM",
    font=("Arial", 24, "bold"),
    bg="#154360",
    fg="white",
    pady=15
)

title.pack(fill="x")


subtitle = tk.Label(
    root,
    text="Integrity & Authenticity Verification",
    font=("Arial", 14, "bold"),
    bg="#EAF2F8",
    fg="#154360"
)
```

```
subtitle.pack(pady=10)

# MESSAGE SECTION
message_frame = tk.LabelFrame(
    root,
    text="1. Enter Message",
    font=("Arial", 12, "bold"),
    bg="#EAF2F8",
    fg="#154360",
    padx=10,
    pady=10
)

message_frame.pack(
    fill="x",
    padx=30,
    pady=5
)

message_entry = tk.Text(
    message_frame,
    height=4,
    font=("Arial", 12),
    wrap="word"
)

message_entry.pack(
    fill="x"
)

# SIGNATURE SECTION
signature_frame = tk.LabelFrame(
    root,
    text="2. Digital Signature",
    font=("Arial", 12, "bold"),
    bg="#EAF2F8",
    fg="#154360",
    padx=10,
    pady=10
)

signature_frame.pack(
    fill="x",
    padx=30,
    pady=5
)

signature_box = tk.Text(
    signature_frame,
    height=5,
    font=("Consolas", 9),
    wrap="word"
)

signature_box.pack(
    fill="x"
)

# HASH SECTION
hash_frame = tk.Frame(
    root,
    bg="#EAF2F8"
)

hash_frame.pack(
    fill="x",
    padx=30,
    pady=5
)
```

```
# Original Hash

left_frame = tk.LabelFrame(
    hash_frame,
    text="Original Message Hash",
    font=("Arial", 11, "bold"),
    bg="#EAF2F8",
    fg="#154360"
)

left_frame.pack(
    side="left",
    fill="both",
    expand=True,
    padx=5
)

hash_box = tk.Text(
    left_frame,
    height=3,
    font=("Consolas", 9),
    wrap="word"
)

hash_box.pack(
    fill="both",
    expand=True,
    padx=5,
    pady=5
)


# Current Hash

right_frame = tk.LabelFrame(
    hash_frame,
    text="Current Message Hash",
    font=("Arial", 11, "bold"),
    bg="#EAF2F8",
    fg="#154360"
)

right_frame.pack(
    side="right",
    fill="both",
    expand=True,
    padx=5
)

current_hash_box = tk.Text(
    right_frame,
    height=3,
    font=("Consolas", 9),
    wrap="word"
)

current_hash_box.pack(
    fill="both",
    expand=True,
    padx=5,
    pady=5
)

# BUTTON SECTION
button_frame = tk.Frame(
    root,
    bg="#EAF2F8"
)
```

```
button_frame.pack(
    pady=15
)


generate_button = tk.Button(
    button_frame,
    text="GENERATE DIGITAL SIGNATURE",
    command=generate_signature,
    font=("Arial", 11, "bold"),
    bg="#2874A6",
    fg="white",
    padx=20,
    pady=10,
    cursor="hand2"
)

generate_button.pack(
    side="left",
    padx=10
)


verify_button = tk.Button(
    button_frame,
    text="VERIFY SIGNATURE",
    command=verify_signature,
    font=("Arial", 11, "bold"),
    bg="#239B56",
    fg="white",
    padx=30,
    pady=10,
    cursor="hand2"
)

verify_button.pack(
    side="left",
    padx=10
)


clear_button = tk.Button(
    button_frame,
    text="CLEAR",
    command=clear_all,
    font=("Arial", 11, "bold"),
    bg="#C0392B",
    fg="white",
    padx=30,
    pady=10,
    cursor="hand2"
)

clear_button.pack(
    side="left",
    padx=10
)
# STATUS
status_label = tk.Label(
    root,
    text="Ready",
    font=("Arial", 12, "bold"),
    bg="#EAF2F8",
    fg="blue"
)

status_label.pack(
    pady=5
)
# RESULT SECTION
```

```
result_label = tk.Label(
    root,
    text="Verification Result",
    font=("Arial", 15, "bold"),
    bg="white",
    fg="black",
    relief="solid",
    bd=1,
    padx=30,
    pady=15
)

result_label.pack(
    fill="x",
    padx=100,
    pady=10
)
# FOOTER
footer = tk.Label(
    root,
    text="RSA + SHA-256 | Digital Signature Practical",
    font=("Arial", 10),
    bg="#EAF2F8",
    fg="#566573"
)

footer.pack(
    side="bottom",
    pady=10
)

# START GUI
root.mainloop()
```

OUTPUT: